

ECE 60146 HW2

Andrea Goh

January 2025

1 Introduction

The goal of this assignment is to gain familiarity with image representations provided by the PIL, Numpy, and PyTorch libraries. This assignment will also explore the concept of data augmentation. Additionally, in this assignment, we will be introduced to the essential components for building an image dataloader to train or test deep neural networks.

2 Pixel Value Scaling, Normalization and Data Augmentation

2.1 Pixel Value Scaling and Normalization

Image data consists of integers ranging from 0 to 255. However, neural networks prefer input data to be floating-point numbers ranging from -1.0 to 1.0. To meet this requirement, pixel value scaling is first used to map the integer image data (0 to 255) to the floating-point range (0, 1.0). Then, pixel value normalization is used to map the scaled pixel values to span the floating-point range (-1.0, 1.0). The normalization step is particularly important for neural networks because it centers the data around zero, which can improve the convergence behavior during training.

2.2 Data Augmentation

In this assignment, the custom dataset that we create starts with 20 base images. These 20 images are augmented to obtain a total of 50 image samples. These augmentations will include random rotations, flips, cropping, and color adjustments. The purpose of data augmentation is to introduce variability within the training dataset, which can reduce overfitting and improve generalization.

2.3 Transformations

Pixel value scaling, normalization and data augmentation can be implemented using the **torchvision.transforms** module. The transformation pipeline used in this assignment for generating the custom dataset is as follows:

```
from torchvision import datasets, transforms

transform_custom = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.RandomResizedCrop(256, scale=(0.9,1)),
    transforms.ToTensor()
    #, transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])
```

Using the code above, the custom dataset can be generated by applying random flips, rotation, and crop to the base image. The **transforms.ToTensor()** transformation applies pixel value scaling. **transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))** applies pixel value normalization, where the first arg, (0.5,0.5,0.5), is subtracted from the corresponding channel pixel value and the result is then divided by the second arg, (0.5,0.5,0.5). In this assignment, the normalization transformation is commented out to allow for better visualization of dataset.

3 Programming Tasks

This assignment uses the following libraries:

```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import os
from PIL import Image
import matplotlib.pyplot as plt
import time
```

3.1 Comparing CIFAR10 with a Custom Dataset

In this section the CIFAR10 dataset in PyTorch is compared with a custom dataset. PyTorch provides the Dataset class to represent datasets. When a dataset is created, it can be loaded into a dataloader, which handles batching, shuffling, and parallel loading of data.

3.1.1 Step 1: Load CIFAR10 Dataset

```
# Step 1: Load CIFAR10 Dataset
transform_cifar = transforms.Compose([transforms.ToTensor()])
cifar10 = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform_cifar)

# Filter 5 classes with 10 images
selected_classes = [1,4,5,7,9]
cifar_indices = [[] for i in range(len(selected_classes))]
for i,c in enumerate(selected_classes):
    cifar_indices[i] = [j for j, (_, label) in enumerate(cifar10) if label==c]
subset_cifar10 = Subset(cifar10, sum([cifar_indices[i][0:10] for i in range(5)],[]))
```

The CIFAR10 dataset is downloaded and transformed using pixel value scaling. Then, a subset of the dataset is created by selecting 5 out of 10 classes and loading 10 images for each of the selected classes.

3.1.2 Step 2: Create Custom Dataset

```
# Step 2: Create Custom Dataset
class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, root, transform):
        self.root = root
        self.image_paths = [os.path.join(root, img) for img in os.listdir(root)]
        self.transform = transform
    def __len__(self):
        return len(self.image_paths)
    def __getitem__(self, index):
        img_path = self.image_paths[index]
        image = Image.open(img_path).convert("RGB")
        transformed_image = self.transform(image)
        return (transformed_image, index)

transform_custom = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor()
    # , transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

custom_dataset = CustomDataset(root='./custom_images', transform=transform_custom)
augmented_images = [custom_dataset[i % len(custom_dataset)][0] for i in range(50)]
```

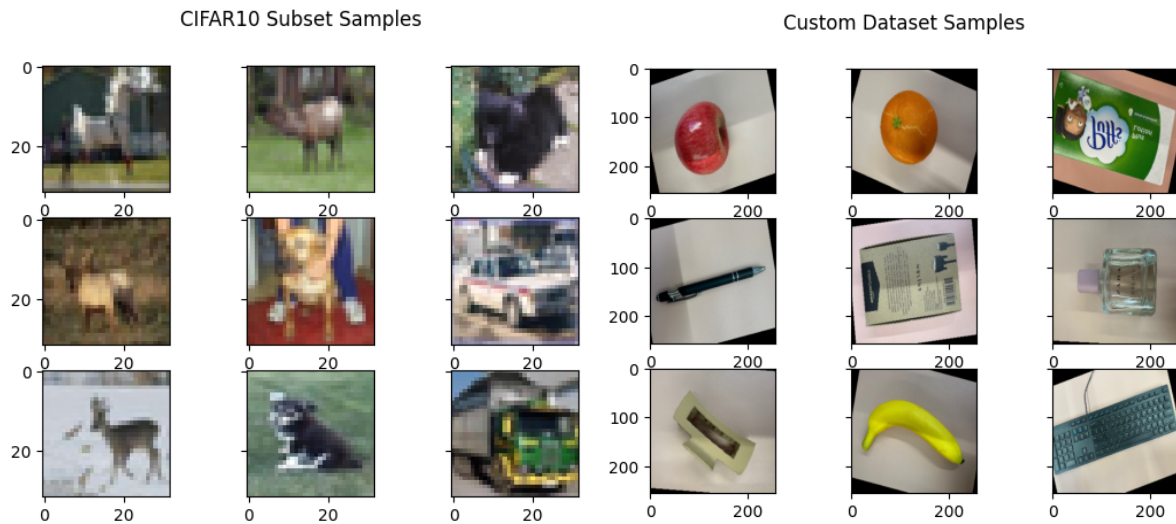


Figure 1: fig: Visualization of Datasets

3.1.3 Step 3: Comparison and Visualization of Datasets

```
# Compare the datasets
print(f'Size of CIFAR10 subset: {len(subset_cifar10)} images')
print(f'Size of Custom Dataset: {len(augmented_images)} images')

# Visualize the dataset
def plot_samples(images, title):
    num = len(images)
    print(num)
    fig, axes = plt.subplots(int(num**0.5), int(num**0.5), sharey=True)
    count = 0
    for i in range(int(num**0.5)):
        for j in range(int(num**0.5)):
            axes[i][j].imshow(transforms.ToPILImage()(images[count]))
            count += 1
    plt.suptitle(title)
    plt.savefig(f'{title}.png', bbox_inches='tight')

batch_size = 9
# CIFAR10 Subset
cifar_loader = DataLoader(subset_cifar10, batch_size, shuffle=True)
batch = next(iter(cifar_loader))
cifar_images = batch[0]
plot_samples(cifar_images, 'CIFAR10 Subset Samples')

# Custom Dataset
custom_loader = DataLoader(augmented_images, batch_size, shuffle=True)
batch = next(iter(custom_loader))
custom_images = batch
plot_samples(custom_images, 'Custom Dataset Samples')
```

The outputs from the code above is as follows:

```
Size of CIFAR10 subset: 50 images
Size of Custom Dataset: 50 images
```

Custom Dataset Samples num_workers = 2

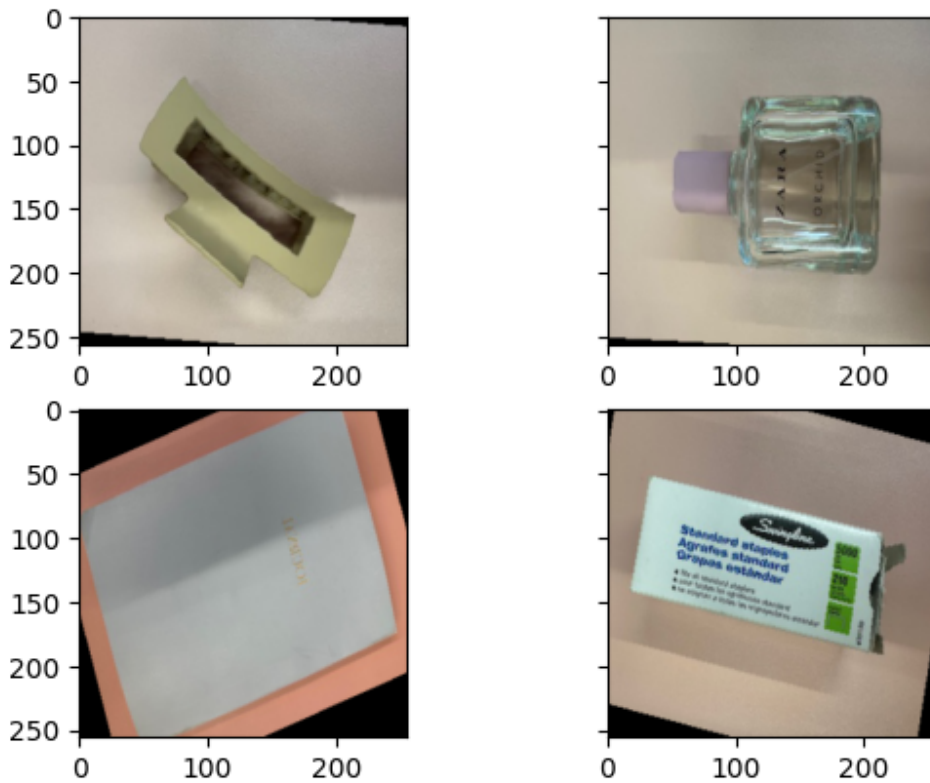


Figure 2: Single batch returned by DataLoader

3.2 Using DataLoader for Parallel Processing

3.2.1 Parallel Processing

```
# Multi-threading
batch_size = 4
custom_loader = DataLoader(augmented_images, batch_size, shuffle=True, num_workers=2)
batch = next(iter(custom_loader))
custom_images = batch
plot_samples(custom_images, 'Custom Dataset Samples num_workers = 2')
```

In this section, the custom dataset, saved in **augmented_images**, is wrapped within a **DataLoader** instance. The batch size is set to 4 and **num_workers** is set to 2, which enables multi-threading. The 4 images from a single batch returned by the **DataLoader** is plotted in figure 2.

batch_size	num_workers	Time taken [seconds]
16	2	0.21245
16	4	0.10931
16	6	0.09412
16	8	0.10165
32	2	0.18302
32	4	0.10625
32	6	0.08296
32	8	0.08344
64	2	0.17574
64	4	0.11248
64	6	0.07127
64	8	0.07527
128	2	0.18804
128	4	0.10635
128	6	0.09520
128	8	0.08057

Table 1: Performance for Parallel Data Loading

3.2.2 Comparing Performance for Parallel Data Loading

```

# Measure time taken to generate 1000 custom images
num_images = 1000
start_time = time.time()
custom_images_1000 = [custom_dataset[i % len(custom_dataset)][0] for i in range(num_images)]
end_time = time.time()
elapsed_time = end_time - start_time
print(f'Time taken to generate {num_images} custom images using __getitem__:
      {elapsed_time} seconds')

# Compare time taken by DataLoader to process 1000 images
batch_size_list = [16, 32, 64, 128]
num_workers_list = [2, 4, 6, 8]

for b_size in batch_size_list:
    for n_workers in num_workers_list:
        data_loader = DataLoader(custom_images_1000, batch_size=b_size,
                                shuffle=True, num_workers=n_workers)
        iterator = iter(data_loader)
        start_time = time.time()
        for i in range(int(num_images/b_size)):
            try:
                batch = next(iterator)
            except StopIteration:
                pass
        end_time = time.time()
        elapsed_time = end_time - start_time
        print(f'batch_size = {b_size}\n')
        print(f'num_workers = {n_workers}\n')
        print(f'Time taken by DataLoader to process all 1000 images:
              {elapsed_time} seconds\n')

```

In this section, 1000 custom images are generated. It takes 2.4449 seconds to load and augment all 1000 images using `__getitem__` in a loop.

Then, the `DataLoader` is used to process all 1000 images with varying `batch_size` and `num_workers` values. The time taken to process the images are measured. The results of all these measurements are shown in the table below:

From the table above, we can see that, as **num_workers** increases, the time taken to process all the data decreases when the batch size is the largest at 128. This trend is observed for all the other batch sizes up until **num_workers** = 6. When **num_workers** is increased beyond 6, the time taken increases. In addition, when the batch size is increased, from 16 to 64, the time taken to process all the data decreases. However, when batch size is further increased to 128, time taken increase. Out of all the measurements taken, the most efficient processing of data occurs when **batch_size** = 64 and **num_workers** = 6.

3.2.3 Maximum RGB values before and after Normalization

```
# Maximum RGB channel values before and after normalization
data_loader = DataLoader(custom_images_1000, batch_size=64, shuffle=True, num_workers=6)
imgs_bef_norm = next(iter(data_loader))
transform_norm = transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
imgs_aft_norm = transform_norm(imgs_bef_norm)
print('Maximum RGB values before normalization: ')
print(torch.amax(imgs_bef_norm, dim=(0, 2, 3)))
print('Maximum RGB values after normalization: ')
print(torch.amax(imgs_aft_norm, dim=(0, 2, 3)))
```

OUTPUT:

```
Maximum RGB values before normalization:
tensor([0.9843, 1.0000, 1.0000])
Maximum RGB values after normalization:
tensor([0.9686, 1.0000, 1.0000])
```

3.3 Exploring Random Seed and Reproducibility

In this section, we explore the use of random seeds to obtain reproducible results. Random seeds are important in deep learning experiments because they enable reproducibility by ensuring that the same sequence of random numbers are generated each time the code runs. This allows us to consistently compare results across different runs of the same experiments.

3.3.1 Without random seed

```
def plot_2_imgs(imgs, title):
    fig, axes = plt.subplots(1, batch_size, sharey=True)
    axes[0].imshow(transforms.ToPILImage()(imgs[0]))
    axes[1].imshow(transforms.ToPILImage()(imgs[1]))
    plt.suptitle(title)
    plt.savefig(f'{title}.png', bbox_inches='tight')
    return

batch_size = 2
data_loader = DataLoader(augmented_images, batch_size=batch_size, shuffle=True)

imgs = next(iter(data_loader))
plot_2_imgs(imgs, '3.4 Without Random Seed First Time')

imgs = next(iter(data_loader))
plot_2_imgs(imgs, '3.4 Without Random Seed Second Time')
```

In the code above, two different trials of the iterator returns two images. The images return each time are different. This is because we set `shuffle=True` in the dataloader. Without a seed, we expect the shuffle to be random and thus, the two trials will result in different images from the same batch. The resulting images are shown in figure 3.3.1

3.4 Without Random Seed First Time

3.4 Without Random Seed Second Time

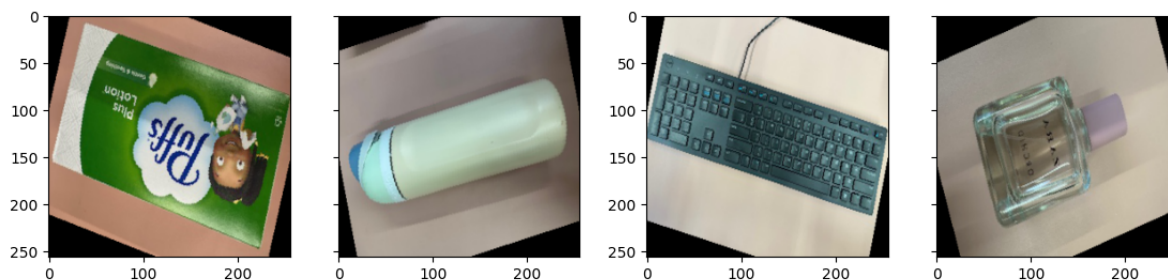


Figure 3: fig: Without Random Seed

3.3.2 With random seed

```
import random
import numpy as np

seed = 60146
# From lecture 3 slide 73
random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
np.random.seed(seed)
torch.backends.cudnn.deterministic=True
torch.backends.cudnn.benchmark=False
os.environ['PYTHONHASHSEED'] = str(seed)

batch_size = 2
data_loader = DataLoader(augmented_images, batch_size=batch_size, shuffle=True)

torch.manual_seed(seed)
imgs = next(iter(data_loader))
plot_2_imgs(imgs, '3.4 With Random Seed First Time')

torch.manual_seed(seed)
imgs = next(iter(data_loader))
plot_2_imgs(imgs, '3.4 With Random Seed Second Time')
```

The same experiment is carried out in the code above. However, this time, the same random seed is set before each time we call the iteration of the dataloader. Due to the manually set seed, we expect the shuffle to be consistent. Thus, the resulting output images will be the same for both trials. This is shown in figure 3.3.2

3.4 With Random Seed First Time

3.4 With Random Seed Second Time

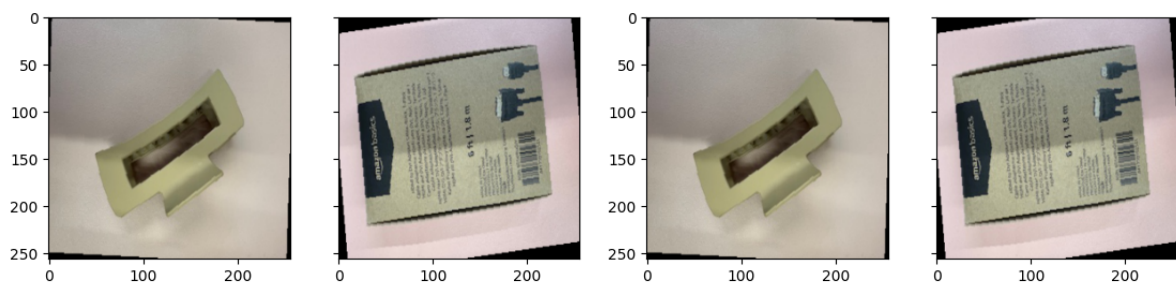


Figure 4: fig: With Random Seed